

CD SECURITY

AUDIT REPORT

Keiko V2
August 2025

Introduction

A time-boxed security review of the **Keiko V2** protocol was done by **CD Security**, with a focus on the security aspects of the application's implementation.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs, and on-chain monitoring are strongly recommended.

About **Keiko V2**

KEIKO is a DeFi lending system allowing users to create vaults with collateral to borrow KEI debt tokens. Main features include vault management with leverage up to 3x via swaps, stability pool for liquidation rewards and protection, yield integration with other DeFi protocols and staking. Automated processes like auto-repay, auto-rate adjustment are also available as additional features.

Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact - the technical, economic, and reputation damage of a successful attack

Likelihood - the chance that a particular vulnerability gets discovered and exploited

Severity - the overall criticality of the risk

Security Assessment Summary

***review commit hash* - 5e597545a81db004d6119fb088ccb652b92d6792**

Scope

The following smart contracts were in scope of the audit:

- `./src/AutoRepayManager.sol`

- `./src/CollateralRiskHelper.sol`
- `./src/RateOperations.sol`
- `./src/RedemptionManager.sol`
- `./src/StabilityPool.sol`
- `./src/StakingModule.sol`
- `./src/SwapRouter.sol`
- `./src/VaultManager.sol`
- `./src/VaultOperations.sol`
- `./src/YieldManager.sol`
- `./src/KEIKO.sol`
- `./src/sKEIKO.sol`
- `./src/AAVEV3Adapter.sol`

The following number of issues were found, categorized by their severity:

- Critical & High: 5 issues
- Medium: 10 issues
- Low: 9 issues

Findings Summary

ID	Title	Severity
[C-01]	`redeemVault` doesn't Withdraw From Adapters	Critical
[C-02]	Staking Dilution via Timed Deposits	Critical
[H-01]	`adjustRateByOperator` Mints Fees to RateOperations Contract Instead of Operator	High
[H-02]	`shortfall` > `collateralAmout` Recovery Not Properly Handled	High
[H-03]	`closeVault` Uses Stale Collateral Amount Information	High
[M-01]	Redemption Cannot Occur for at least 14 Days from Deployment	Medium
[M-02]	Malicious User Can Hold StakingModule Hostage	Medium
[M-03]	Inconsistent Yield Gain Accounting in StabilityPool Withdrawals	Medium
[M-04]	AaveV3 hasn't been deployed to hyperEVM	Medium
[M-05]	No Function to Claim Aave Rewards from Rewards Controller	Medium
[M-06]	Incorrect Increment in `_sendGainsToDepositor`	Medium
[M-07]	Single Adapter Failure Blocks All Yield Claims	Medium
[M-08]	RSnapshot Desynchronization in `withdraw()` Leads to Yield Theft	Medium
[M-09]	Linear approximation used for the fractional half-life period of the exponential decay calculation is inaccurate	Medium

ID	Title	Severity
[M-10]	Violation of Economic Invariant	Medium
[L-01]	Yield Distribution Percentage Sum Not Checked	Low
[L-02]	Unbounded Withdrawal Requests in `StakingModule`	Low
[L-03]	Incorrect Negative Collateral Check	Low
[L-04]	Missing Min Debt Amount Check in Various Cases	Low
[L-05]	Inconsistent Total Protocol Debt Tracking	Low
[L-06]	Changing `withdrawCooldown` impact pending withdrawals	Low
[L-07]	Immutable Aave Pool Address	Low
[L-08]	`emergencyWithdraw`'s Catch Logic Could be Dangerous`	Low
[L-09]	Rate Adjustments with 0 `targetDebtAhead` can be created	Low

Detailed Findings

[C-01] `redeemVault` doesn't Withdraw From Adapters

Severity

Impact:

High

Likelihood:

High

Description

In `redeemVault`, `_executeCollateralRedemption` calls `_redeemFromSingleVault` which handles collateral redemption from the vaults individually.

Following this function, based on the provided tags, we can see that:

1. The total collateral a vault has is retrieved from the vault manager via `getVaultData`. This `currentCollateral` param holds all the collateral in the vault, both deployed to adapters and idle in VaultOperations.
2. The `newCollateral` amount that the vault will have after the redemption is calculated by subtracting the `collateralAmount` to redeem from the vault from `currentCollateral`. In case `collateralAmount` >= `currentCollateral`, `newCollateral` will be set to 0.

3. The `newDebt` is set as the vault collateral amount which is now 0. This is very important to note because a vault with 0 collateral is marked as non existent, based on various checks across the contracts.
4. The collateral transfer is then handled in `redeemVault` which transfers the collateral to the redeemer via `transferCollateralForRedemption`.

```
    function _redeemFromSingleVault(
        address collateral,
        address vaultOwner,
        uint256 keiAmount,
        uint256 collateralAmount,
        address prevId,
        address nextId
    ) internal {
        // Get current vault data and calculate new amounts
        (uint256 currentCollateral, uint256 currentDebt,) =
            IVaultManager(vaultManager).getVaultData(collateral,
vaultOwner); //<@

        uint256 newDebt = currentDebt > keiAmount ? currentDebt -
keiAmount : 0;
        uint256 newCollateral = currentCollateral > collateralAmount ?
            currentCollateral - collateralAmount : 0;//<@

        // Update vault data with current rate maintained
        IVaultManager(vaultManager).adjustVaultData(//<@
            collateral,
            vaultOwner,
            newCollateral,
            newDebt,
            IVaultManager(vaultManager).getVaultInterestRate(collateral,
vaultOwner)
        );

        // Handle vault sorting based on remaining debt
        if (newDebt > 0) {
            IVaultSorter(vaultSorter).reInsertVault(
                collateral,
                vaultOwner,
                IVaultManager(vaultManager).calculateARS(collateral,
vaultOwner),
                prevId,
                nextId
            );
        } else {
            IVaultSorter(vaultSorter).removeVault(collateral, vaultOwner);
        }

        // Emit events with calculated remaining amounts
        uint256 vaultARS = newDebt > 0 ?
IVaultManager(vaultManager).calculateARS(collateral, vaultOwner) : 0;
```

```
        emit VaultRedeemed(
            vaultOwner,
            collateral,
            msg.sender,
            keiAmount,
            collateralAmount,
            newDebt,
            newCollateral,
            vaultARS
        );

        if (newDebt == 0 && newCollateral > 0) {
            emit VaultFullyRedeemed(vaultOwner, collateral,
newCollateral);
        }
    }
```

```
    function redeemVault(
//...
        for (uint256 i = 0; i < sortedCollaterals.length && remainingKei >
0; i++) {
            uint256 availableDebt =
IVaultOperations(vaultOperations).totalDebt(sortedCollaterals[i]);

            if (availableDebt > 0) {
                results[bundleIndex] = _executeCollateralRedemption(
                    sortedCollaterals[i],
                    remainingKei < availableDebt ? remainingKei :
availableDebt,
                    effectiveRate,
                    prevId,
                    nextId
                );

                remainingKei -= results[bundleIndex].keiAmount;
                bundleIndex++;
            }
        }

        // Finalize redemption
        uint256 totalKeiRedeemed = actualRedemptionAmount - remainingKei;
        _updateBaseRate(totalKeiRedeemed, totalProtocolDebt);
        IERC20(debtToken).burn(redeemer, totalKeiRedeemed);

        // Transfer collaterals to redeemer
        for (uint256 i = 0; i < bundleIndex; i++) {
            if (results[i].collateralAmount > 0) {

IVaultOperations(vaultOperations).transferCollateralForRedemption(//<@
                    results[i].collateralToken,
                    results[i].collateralAmount
```

```
                );
                IERC20(results[i].collateralToken).transfer(redeemer,
results[i].collateralAmount);
            }
        }

        assembly { mstore(results, bundleIndex) }

        emit CrossCollateralRedemptionExecuted(redeemer, totalKeiRedeemed,
effectiveRate, bundleIndex);

        return results;
    }
```

Now, upon collateral deployment to a yield adapter, we can see that, collateral deployed to yield, although tracked separately, is not removed from total vault collateral as intended by protocol operations.

```
    function deployCollateralToYield(address vaultCollateral, uint256
amount, address adapter) external {
        (uint256 collateralAmount, , ) =
manageDebtInterest(vaultCollateral, msg.sender);

        if (collateralAmount == 0) revert VaultDoesntExist();
        if (amount == 0) revert AmountMustBeGreaterThanZero();
        if (amount > collateralAmount) revert
AmountExceedsAvailableCollateral();

        IERC20(vaultCollateral).approve(yieldManager, amount);

        // Deploy collateral to yield strategy
        bool success = IYieldManager(yieldManager).deployCollateral(
            vaultCollateral,
            msg.sender,
            amount,
            adapter
        );

        if (!success) revert DeploymentFailedBufferRequirements();

        emit CollateralDeployedToYield(msg.sender, vaultCollateral,
amount);
    }

    function deployCollateral(
        address vaultCollateral,
        address vaultOwner,
        uint256 amount,
        address adapter
    )
        external
        override
```

```
        onlyVaultOperations
        returns (bool)
    {
        require(isAdapterApproved[vaultCollateral][adapter], "Adapter not
approved");
        require(adapterInfo[vaultCollateral][adapter].active, "Adapter not
active");

        uint256 deployableAmount =
calculateDeployableAmount(vaultCollateral, vaultOwner);

        require(deployableAmount > 0, "No deployable amount available");
        require(amount <= deployableAmount, "Amount exceeds deployable
limit");

        userAdapterDeposits[vaultOwner][vaultCollateral][adapter] +=
amount;

        IERC20(vaultCollateral).transferFrom(vaultOperations,
address(this), amount);
        IERC20(vaultCollateral).approve(adapter, amount);
        IYieldAdapter(adapter).deposit(vaultCollateral, amount,
vaultOwner);

        emit CollateralDeployed(vaultOwner, vaultCollateral, amount);
        return true;
    }
```

The impacts of this are as follows:

1. Redemption fails due to insufficient collateral amount, if a large amount of collateral is deployed to yield and a larger amount is requested to be redeemed.
2. More likely and more serious - vaults with collateral amount less than amount to redeem will have their collateral amount updated to 0. Once this is done, their vaults will be treated as non-existent. We can track this by checking for the `VaultDoesntExist` error across the contracts, but essentially, they cannot close the vault, retrieve their collateral from adapter or even be liquidated if they have debt available. Note that they're also not removed from available vaults. Since the deployed collateral will be irretrievably, the actual token balance of VaultOperations will end up being less than total collateral which will inadvertently lead to insolvency.

Recommendations

Compare collateral available in vault with amount to redeem and if its less, execute a yield withdrawal before redeeming collateral.

[C-02] Staking Dilution via Timed Deposits

Severity Critical

Impact:

High

Likelihood:

High

Description

An attacker can exploit the time lag between yield accumulation and its subsequent processing to capture a disproportionate share of staking rewards, thereby diluting the returns for all other stakers.

This is achieved by making a large KEIKO deposit immediately before the accumulated KEI yield is swapped for KEIKO and added to the staking pool.

The attack is made more potent by the public accessibility of the `processAccumulatedYield` function, which allows the attacker to trigger the yield processing event at a time of their choosing to maximize their profit. The core of the vulnerability lies in the absence of a mechanism to align reward eligibility with the duration of the stake, such as a timelock or time-weighted share system.

Attack Path

1. **Yield Accumulation Phase:** Over time, the `VaultOperations` contract directs a portion of the protocol's yield (in the form of KEI debt tokens) to the `StakingModule` via the `receiveYield` function. This yield accumulates in the `pendingKEIYield` state variable.

   Relevant Code (`StakingModule.sol`):

```
    function receiveYield(uint256 _yieldAmount) external override {
        require(_yieldAmount > 0, "Amount must be greater than 0");
        IERC20(debtToken).transferFrom(msg.sender, address(this),
_yieldAmount);
        pendingKEIYield += _yieldAmount;
        emit YieldReceived(_yieldAmount);
    }
```

2. **Monitoring Phase:** A sophisticated attacker continuously monitors the public `pendingKEIYield` variable. Once the accumulated amount reaches or exceeds the `minSwapAmount` (default 100 KEI), the contract is ripe for the attack.

3. **Exploitation Phase (Single Transaction Execution):** The attacker executes a transaction that front-runs any organic call to `processAccumulatedYield`, or they simply call it themselves.

Step 1: Large Deposit: The attacker first calls the `stake` function to deposit a large amount of KEIKO tokens. They receive sKEIKO shares calculated at the current exchange rate, which does *not* yet factor in the `pendingKEIYield`.

Relevant Code (`StakingModule.sol`):

```
        function stake(uint256 _amount) external {
            // ...
            uint256 sharesToMint = getSharesByPooledKEIKO(_amount);
            // ...
            totalPooledKEIKO += _amount;
            totalShares += sharesToMint;
            IERC20(sKEIKO).mint(msg.sender, sharesToMint);
            // ...
        }
```

Step 2: **Trigger Yield Processing:** In the same transaction or immediately following it, the attacker calls the public processAccumulatedYield function. This function swaps the `pendingKEIYield` for KEIKO and adds the resulting amount to the `totalPooledKEIKO`.

Relevant Code (`StakingModule.sol`):

```
        function processAccumulatedYield(...) external returns (uint256) {
            // ...
            uint256 keikoReceived =
ISwapRouter(swapRouter).executeKEItoKEIKOSwap(...);
            // ...
            totalPooledKEIKO += keikoReceived; // Share value appreciates
here
            // ...
        }
```

This step increases the value of *all* sKEIKO shares, including the ones the attacker just minted.

4. **Profit Realization Phase:** The attacker can now immediately call the `unstake` function. They will receive back their principal plus a share of the processed yield that they did not contribute to generating.

   Relevant Code (`StakingModule.sol`):

   ```
   function unstake(uint256 _shares) external returns (uint256
   keikoAmount) {
       // ...
       keikoAmount = getPooledKEIKOByShares(_shares); // Calculated with
   inflated totalPooledKEIKO
       // ...
   }
   ```

4. Impact

The economic impact is a direct and quantifiable loss for legitimate stakers. The attacker's profit is a parasitic drain on the yield that should have been distributed to long-term supporters of the protocol.

- **Example Scenario:**
  - Total Pooled KEIKO: 1,000,000
  - Total Shares: 1,000,000 (Exchange Rate: 1 sKEIKO = 1 KEIKO)
  - Pending KEI Yield: 50,000 (after swap)
  - Attacker stakes 1,000,000 KEIKO, minting 1,000,000 new shares.
  - New State: 2,000,000 KEIKO, 2,000,000 shares.
  - `processAccumulatedYield` is called. `totalPooledKEIKO` becomes 2,050,000.
  - New Exchange Rate: 1 sKEIKO = 1.025 KEIKO.
  - The attacker now owns 50% of the pool and has effectively claimed 25,000 KEIKO of the yield, despite having their funds at risk for only a few blocks.
  - Long-term stakers, who should have received the full 50,000 KEIKO yield (a 5% return), now only see a 2.5% return. Their yield has been diluted by 50%.

This can be repeated every time the yield is processed, leading to a significant and continuous drain on the protocol's value accrual.

Recommendation

The vulnerability can be mitigated by ensuring that rewards are allocated based on the duration of a stake. The following solutions are recommended:

1. **Implement Time-Weighted Shares:**

2. **Introduce a Staking Cooldown/Timelock:**

[H-01] `adjustRateByOperator` Mints Fees to RateOperations Contract Instead of Operator

Severity

Impact:

High

Likelihood:

Medium

Description

`adjustRateByOperator` mints fees `msg.sender` which is RateOperations contract. The contract however doesn't have a way to transfer the minted fees to the operator/caller, or any other ways to retrieve tokens, locking them indefinitely.

```
    function adjustRateByOperator(
        address vaultCollateral,
        address vaultOwner,
        uint256 newRate,
```

```
        uint256 feeAmount
    ) external onlyRateOperations {
        (uint256 collateralAmount, uint256 debtAmount, ) =
            IVaultManager(vaultManager).getVaultData(vaultCollateral,
vaultOwner);

        uint256 newDebtAmount = debtAmount + feeAmount;
        IVaultManager(vaultManager).adjustVaultData(
            vaultCollateral,
            vaultOwner,
            collateralAmount,
            newDebtAmount,
            newRate
        );

        totalDebt[vaultCollateral] += feeAmount;
        totalProtocolDebt += feeAmount;

        IERC20(debtToken).mint(msg.sender, feeAmount);//<@
    }
```

```
    function adjustVaultRate(
        address user,
        address collateral,
        uint256 newRate,
        address prevId,
        address nextId,
        uint256 depthLimit
    ) external {
        RateAdjustmentConfig storage config = adjustmentConfigs[user]
[collateral];
        require(config.enabled, "Rate adjustment not enabled for this
vault");

        (uint256 collateralAmount, uint256 debtAmount, uint256
currentRate) =
            IVaultManager(vaultManager).getVaultData(collateral, user);

        require(collateralAmount > 0, "Vault doesn't exist");
        require(debtAmount > 0, "Cannot adjust rate on zero-debt vault");

        bool isWhitelisted = _isOperatorWhitelisted(config, msg.sender);

//...
        // Calculate fee
        uint256 feeAmount = _calculateFee(debtAmount,
config.feePercentage);

        // Execute rate adjustment via VaultOperations
        IVaultOperations(vaultOperations).adjustRateByOperator(
            collateral,
```

```
            user,
            finalRate,
            feeAmount
        );

        // Update vault position in sorted list
        uint256 newARS =
IVaultManager(vaultManager).calculateARS(collateral, user);
        IVaultSorter(vaultSorter).reInsertVault(collateral, user, newARS,
prevId, nextId);

        emit RateAdjustedByOperator(user, collateral, msg.sender,
currentRate, finalRate, feeAmount, isWhitelisted);
    }
```

Rate fees are minted to wrong address

Recommendations

Transfer minted fees to operator/caller.

[H-02] `shortfall` > `collateralAmount` Recovery Not Properly Handled

Severity

Impact:

High

Likelihood:

High

Description

In `YieldManager.sol`, the `attemptRecovery` function allows a user to recover collateral previously marked as pending (failed withdrawals). The function checks if the user's vault still exists by calling `IVaultManager(vaultManager).getVaultData`. If the vault has been closed (`collateralAmount == 0`), it sends any recovered tokens directly to the user.

This logic is flawed because the protocol absorbs the loss from this pending collateral via the `adjustVaultForYieldShortfall` logic while capping user vault collateral to 0.

```
    function adjustVaultForYieldShortfall(
        address vaultCollateral,
        address vaultOwner,
        uint256 shortfallAmount
    ) external onlyYieldManager {
```

```
        Vault storage userVault = userVaults[vaultOwner][vaultCollateral];

        // Calculate new collateral amount after shortfall
        uint256 newCollateralAmount = userVault.collateral >
shortfallAmount ?
            userVault.collateral - shortfallAmount : 0;//<@

        userVault.collateral = newCollateralAmount;


IVaultOperations(vaultOperations).updateTotalCollateral(vaultCollateral, -
int256(shortfallAmount));

        emit VaultCollateralAdjusted(vaultOwner, vaultCollateral, false,
shortfallAmount);
    }
```

But when user calls `attemptRecovery`, the entire collateral is sent to the user without updating the total protocol collateral. So the protocol absorbed loss is not compensated for.

```
    function attemptRecovery(address vaultCollateral, address vaultOwner)
external returns (uint256) {
//...
        }

        if (totalRecovered > 0) {
            // Check if vault exists
            (uint256 collateralAmount, , ) =
IVaultManager(vaultManager).getVaultData(vaultCollateral, vaultOwner);

            if (collateralAmount == 0) {//<@
                // Vault doesn't exist - send tokens directly to user
                IERC20(vaultCollateral).transfer(vaultOwner,
totalRecovered);
            } else {
                // Vault exists - send to VaultOperations and adjust vault
data
                IERC20(vaultCollateral).transfer(vaultOperations,
totalRecovered);
                IVaultManager(vaultManager).adjustVaultForYieldRecovery(
                    vaultCollateral,
                    vaultOwner,
                    totalRecovered
                );
            }

            emit CollateralRecovered(vaultOwner, vaultCollateral,
totalRecovered);
        }
```

```
        return totalRecovered;
    }
```

If multiple significant shortfalls occur, this could lead to protocol insolvency, where the value of the internal `totalCollateral` accounting is significantly lower than the real value of assets backing the system, potentially causing a bank run or breaking core protocol invariants.

Consider this rough scenario :

1. Total protocol collateral = 1000;
2. User has vault with 100 but upon retrieval, experiences a shortfall of 200;
3. So his collateralAmount is reduced to 0 while total protocol collateral is reduced to 800;
4. User calls `attemptRecovery` and ends up receiving the 200 shortfall.
5. Protocol is still default 200.

Recommendations

Track how much of the shortfall exceeds the collateral amount and use the excess to update total protocol collateral. And send it to vault operations.

[H-03] `closeVault` Uses Stale Collateral Amount Information

Severity

Impact:

Medium

Likelihood:

High

Description

Users can deploy a portion of their collateral to a yield, e.g aave. This deployed collateral may either be worth more than initial amount deployed, or less in case of a shortfall. WHen closing vault however, this potential change in collateral amount is not accounted for.

`closeVault` retrieves collateral from the yield via `retrieveCollateral` which adjusts the user and protocol's vault data based on if the amount retrieved exceeds or is less than the expected amount. This is done via `adjustVaultForYieldShortfall` or `adjustVaultForYieldRecovery` which also update the new collateral amount of the vault.

This now, newly updated collateral amount is what should be used during closure but the function doesn't. It still sends the initial collateral amount to the user which can lead to two major impacts.

1. Loss of funds for the user if the deployed collateral, upon withdrawal ends up increasing in value. This is because, even though `collateralAmount` has been increased in

`adjustVaultForYieldRecovery`, stale `collateralAmount` is being used. This can be mitigated for users by claiming yield, or directly retrieving collateral first, before closing vault.

2. An avenue to rob the protocol of funds in case of a shortfall. `collateralAmount` will be reduced in `adjustVaultForYieldShortfall`, while `userAdapterPending` is increased. But because stale `collateralAmount` is being used, the user receives the collateral amount in full. He can then call `attemptRecovery` which will transfer pending amount to him, so he gets paid double.

```
    function closeVault(address vaultCollateral) external {
        uint256 deployedAmount =
IYieldManager(yieldManager).getDeployedCollateral(vaultCollateral,
msg.sender);
        (uint256 collateralAmount, uint256 debtAmount,) =
manageDebtInterest(vaultCollateral, msg.sender);//<@

        if (deployedAmount > 0)
IYieldManager(yieldManager).retrieveCollateral(vaultCollateral,
msg.sender);//<@
        if (collateralAmount == 0) revert VaultDoesntExist();

        if (debtAmount > 0) {
            if (IERC20(debtToken).balanceOf(msg.sender) < debtAmount)
revert InsufficientDebtTokenBalance();

            totalDebt[vaultCollateral] -= debtAmount;
            totalProtocolDebt -= debtAmount;
        }

        totalCollateral[vaultCollateral] -= collateralAmount;

        activeVaults -= 1;
        lastDebtUpdateTime[msg.sender][vaultCollateral] = 0;

        // Clear vault data and remove from sorter
        IVaultManager(vaultManager).adjustVaultData(vaultCollateral,
msg.sender, 0, 0, 0);
        IVaultSorter(vaultSorter).removeVault(vaultCollateral,
msg.sender);

        if (debtAmount > 0) {
            IERC20(debtToken).burn(msg.sender, debtAmount);
        }

        IERC20(vaultCollateral).transfer(msg.sender, collateralAmount);

        emit VaultClosed(msg.sender, vaultCollateral, collateralAmount,
debtAmount);
    }
```

```
    function retrieveCollateral(address vaultCollateral, address
vaultOwner)
        external
        override
        onlyVaultOperations
        returns (uint256)
    {
        address[] memory adapters = approvedAdapters[vaultCollateral];
        uint256 totalRetrieved = 0;
        uint256 totalExpected = 0;

        for (uint i = 0; i < adapters.length; i++) {
            address adapter = adapters[i];
            uint256 deployedAmount = userAdapterDeposits[vaultOwner]
[vaultCollateral][adapter];

            if (deployedAmount == 0) continue;

            totalExpected += deployedAmount;

            userAdapterDeposits[vaultOwner][vaultCollateral][adapter] = 0;

            // UPDATED: Pass vaultOwner to adapter with new interface
            uint256 actualWithdrawn = IYieldAdapter(adapter).withdraw(
                vaultCollateral,
                deployedAmount,
                vaultOwner,        // NEW: Pass the vault owner
                vaultOperations    // Recipient stays as vaultOperations
            );

            totalRetrieved += actualWithdrawn;

            if (actualWithdrawn < deployedAmount) {
                uint256 shortfall = deployedAmount - actualWithdrawn;//<@

                userAdapterPending[vaultOwner][vaultCollateral][adapter]
+= shortfall;//<@

                emit CollateralRetrievalShortfall(vaultOwner,
vaultCollateral, shortfall);
            }

            // Handle excess if any (unusual but possible)
            if (actualWithdrawn > deployedAmount) {
                uint256 excess = actualWithdrawn - deployedAmount;//<@
                emit ExcessCollateralRetrieved(vaultOwner,
vaultCollateral, excess);
            }
        }

        // Handle total shortfall if any
        if (totalRetrieved < totalExpected) {
            uint256 totalShortfall = totalExpected - totalRetrieved;
```

```
            _adjustVaultDataForShortfall(vaultOwner, vaultCollateral,
totalShortfall);//<@
        }

        // Handle total excess if any
        if (totalRetrieved > totalExpected) {
            uint256 totalExcess = totalRetrieved - totalExpected;
            _adjustVaultDataForRecovery(vaultOwner, vaultCollateral,
totalExcess);//<@
        }

        emit CollateralRetrieved(vaultOwner, vaultCollateral,
totalRetrieved);
        return totalRetrieved;
    }
```

```
    function adjustVaultForYieldShortfall(
        address vaultCollateral,
        address vaultOwner,
        uint256 shortfallAmount
    ) external onlyYieldManager {
        Vault storage userVault = userVaults[vaultOwner][vaultCollateral];

        // Calculate new collateral amount after shortfall
        uint256 newCollateralAmount = userVault.collateral >
shortfallAmount ?
            userVault.collateral - shortfallAmount : 0;

        userVault.collateral = newCollateralAmount;//<@


IVaultOperations(vaultOperations).updateTotalCollateral(vaultCollateral, -
int256(shortfallAmount));//<@

        emit VaultCollateralAdjusted(vaultOwner, vaultCollateral, false,
shortfallAmount);
    }

    /**
     * @notice Updates vault data to reflect collateral recovery from
yield strategy
     * @param vaultCollateral Collateral token address
     * @param vaultOwner Owner of the vault
     * @param recoveredAmount Amount of collateral that was recovered
     */
    function adjustVaultForYieldRecovery(
        address vaultCollateral,
        address vaultOwner,
        uint256 recoveredAmount
    ) external onlyYieldManager {
        Vault storage userVault = userVaults[vaultOwner][vaultCollateral];
```

```
        // Calculate new collateral amount after recovery
        uint256 newCollateralAmount = userVault.collateral +
recoveredAmount;//<@

        userVault.collateral = newCollateralAmount;


IVaultOperations(vaultOperations).updateTotalCollateral(vaultCollateral,
int256(recoveredAmount));//<@

        emit VaultCollateralAdjusted(vaultOwner, vaultCollateral, true,
recoveredAmount);
    }
```

Recommendations

Call `manageDebtInterest` after retrieving collateral, not before.

[M-01] Redemption Cannot Occur for at least 14 Days from Deployment

Severity

Impact:

Low

Likelihood:

High

Description

`RedemptionManager.sol` is meant to burn KEI tokens in the `redeemVault` function, but cannot do so as KEI.sol requires burning to be done by a whitelisted party. But whitelisiting an address on KEI.sol has a 14 day timelock from time of request. So within that time frame, users will not be able to redeem.

```
    function redeemVault(
        address redeemer,
        uint256 keiAmount,
        uint256 maxFeePercentage,
        address prevId,
        address nextId
    ) external returns (CollateralBundle[] memory results) {
///...
        // Finalize redemption
        uint256 totalKeiRedeemed = actualRedemptionAmount - remainingKei;
        _updateBaseRate(totalKeiRedeemed, totalProtocolDebt);
```

```
        IERC20(debtToken).burn(redeemer, totalKeiRedeemed);//<@

        // Transfer collaterals to redeemer
        for (uint256 i = 0; i < bundleIndex; i++) {
            if (results[i].collateralAmount > 0) {

IVaultOperations(vaultOperations).transferCollateralForRedemption(
                    results[i].collateralToken,
                    results[i].collateralAmount
                );
                IERC20(results[i].collateralToken).transfer(redeemer,
results[i].collateralAmount);
            }
        }

        assembly { mstore(results, bundleIndex) }

        emit CrossCollateralRedemptionExecuted(redeemer, totalKeiRedeemed,
effectiveRate, bundleIndex);

        return results;
    }
```

Recommendations

Might need to upgrade `KEI.sol` to allow burning by `RedemptionManager.sol`, or include information about this possibility of this occurring in protocol documentation.

[M-02] Malicious User Can Hold StakingModule Hostage

Severity

Impact:

High

Likelihood:

Low

Description

`getSharesByPooledKEIKO` returns shares at a 1 to 1 ratio for the first staker which can be taken advantage of by malicious users to execute a pool inflation attack.

```
    function getSharesByPooledKEIKO(uint256 _keikoAmount) public view
returns (uint256) {
        if (totalPooledKEIKO == 0) {
```

```
            return _keikoAmount; // 1:1 for first staker //<@
        }
        return (_keikoAmount * totalShares) / totalPooledKEIKO;
    }
```

The process is as follows:

1. User stakes one wei of KEIKO and his minted one share of sKEIKO. `totalPooledKEIKO` is set to 1 and `totalShares` is set to 1.
2. He transfers large sums of KEI to contract via `receiveYield` function which increases `pendingKEIYield`.
3. He calls `processAccumulatedYield` to swap and increase `totalPooledKEIKO` to a large amount.
4. At this point, subsequent stakers will not be able to stake as `getSharesByPooledKEIKO` will return 0 due to round down.
5. No one can stake at this point.
6. The user now waits for subsequent calls to `receiveYield` and `processAccumulatedYield` that will increase `totalPooledKEIKO` keiko and subsequently, the only share of the user will increase in value. This is bound to happen whenever `mintVaultsInterest` is called in `VaultOperations.sol` or an unsuspecting user calls `receiveYield` in `StakingModule.sol`.

Recommendations

Add access control to `receiveYield` and `processAccumulatedYield` functions in `StakingModule.sol`

[M-03] Inconsistent Yield Gain Accounting in StabilityPool Withdrawals

Severity

Impact:

Medium

Likelihood:

Medium

Description

In withdraw, the `newDeposit` is calculated as `compoundedDeposit - debtTokensToWithdraw` but yield gain is not factored in. This breaks symmetry in `deposit` and `withdrawForAutoRepay` functions as they both factor in yield gain when updating the available deposited amount.

In the `StabilityPool` contract, the handling of yield gains during deposit and withdrawal operations shows inconsistency, particularly when users choose not to claim their yield during withdrawals. This breaks

the symmetry expected in deposit and withdrawal behaviors, leading to potential accounting desynchronizations.

Specifically:

- In the `deposit` function, when `_claimYield` is false, the yield gain is compounded into the `newDeposit` and `newTotalDeposits`, and `yieldGainsOwed` is reduced by the `currentYieldGain` regardless of claiming:

```
uint256 newDeposit = compoundedDeposit + _amount + (currentYieldGain -
yieldToSend);
uint256 newTotalDeposits = totalDebtTokenDeposits + _amount +
(currentYieldGain - yieldToSend);
...
if (currentYieldGain > 0) {
    yieldGainsOwed -= currentYieldGain;
}
```

- In the `withdrawForAutoRepay` function (which always compounds yield), the yield is added to `newDeposit` and `totalDebtTokenDeposits`, and `yieldGainsOwed` is reduced:

```
uint256 newDeposit = compoundedDeposit - actualWithdrawal +
currentYieldGain;
...
if (currentYieldGain > 0) {
    yieldGainsOwed -= currentYieldGain;
}
totalDebtTokenDeposits = totalDebtTokenDeposits - actualWithdrawal +
currentYieldGain;
```

- However, in the `withdraw` function when `_claimYield` is false, the yield gain is **not** added to `newDeposit` or `totalDebtTokenDeposits`, and `yieldGainsOwed` is **not** reduced. Instead, only the G snapshot is preserved to allow future claiming:

```
 function withdraw(uint256 _amount, address[] calldata _assets, bool
_claimYield) external {
      uint256 initialDeposit = deposits[msg.sender];
      require(initialDeposit != 0, "StabilityPool: User must have a
non-zero deposit");

      // Get collateral gains
      (address[] memory assets, uint256[] memory amounts) =
getDepositorGains(msg.sender, _assets);

      uint256 compoundedDeposit =
getCompoundedDebtTokenDeposits(msg.sender);
      uint256 debtTokensToWithdraw = VaultMath._min(_amount,
```

```
compoundedDeposit);
      uint256 loss = initialDeposit - compoundedDeposit; // Needed
only for event log

      // First check if we're claiming yield and calculate it before
any updates
      uint256 yieldToClaim = 0;
      if (_claimYield) {
          yieldToClaim = getDepositorYieldGain(msg.sender);
      }

      // Update deposit
      uint256 newDeposit = compoundedDeposit -
debtTokensToWithdraw;//<@

      // Update deposit and snapshots, but preserve G if not claiming
yield
      if (!_claimYield) {
          // Store old G value before update
          uint256 oldG = depositSnapshots[msg.sender].G;

          // Update all snapshots
          _updateDepositAndSnapshots(msg.sender, newDeposit);

          // Restore old G to preserve yield
          if (newDeposit > 0) {
              depositSnapshots[msg.sender].G = oldG;
          }
      } else {
          // Normal update when claiming yield
          _updateDepositAndSnapshots(msg.sender, newDeposit);
      }

      // If claiming yield, handle it
      if (_claimYield && yieldToClaim > 0) {//<@
          // Decrease owed yield
          yieldGainsOwed -= yieldToClaim;//<@

          // Send yield
          IERC20(debtToken).transfer(msg.sender, yieldToClaim);
          emit YieldClaimed(msg.sender, yieldToClaim);
      }

      // Update total deposits
      totalDebtTokenDeposits = totalDebtTokenDeposits -
debtTokensToWithdraw;

      // Send debt tokens to withdraw
      IERC20(debtToken).transfer(msg.sender, debtTokensToWithdraw);

      // Send collateral gains
      _sendGainsToDepositor(msg.sender, assets, amounts);

      emit UserDepositChanged(msg.sender, newDeposit);
```

```
        emit UserWithdrawal(msg.sender, debtTokensToWithdraw);
        emit GainsWithdrawn(msg.sender, assets, amounts, loss); // loss
required for event log
        emit
StabilityPoolDebtTokenBalanceUpdated(totalDebtTokenDeposits);

        require(totalDebtTokenDeposits >= MIN_DEBT_TOKEN_IN_SP,
"Withdrawal must leave at least MIN_DEBT_TOKEN_IN_SP");
    }
```

This inconsistency means that during a withdrawal without claiming yield, the protocol does not compound the yield into the deposit balance, unlike in deposit (no claim) and autorepay scenarios. As a result:

- The user's deposit shrinks without including the accrued yield, but the yield remains claimable later via separate functions like `claimYield`.
- `yieldGainsOwed` is not reduced when not claiming, leading to the protocol tracking more owed yields than reflected in `totalDebtTokenDeposits`.
- This can cause a desynchronization where the total tokens held by the pool plus owed yields exceed the actual balances, potentially leading to shortfalls if many users withdraw without claiming.
- Users might exploit this by withdrawing without claiming, reducing their deposit exposure while keeping yield owed, then claiming later, effectively getting yield on a reduced principal.

Recommendations

To resolve this inconsistency:

- In `withdraw` when `_claimYield` is false, compound the yield into `newDeposit` and `totalDebtTokenDeposits`, and reduce `yieldGainsOwed` by the yield amount, similar to `deposit` (no claim) and `withdrawForAutoRepay`.

[M-04] AaveV3 hasn't been deployed to hyperEVM

Severity

Impact:

Low

Likelihood:

High

Description

From Aave docs, hyperEVM is not listed among the supported networks. So AaveV3Adapter.sol cannot be used.

Recommendations

Contact aave team for relevant deployments, or make use of a fork if available.

[M-05] No Function to Claim Aave Rewards from Rewards Controller

Severity

Impact:

Medium

Likelihood:

Medium

Description

From Aave Docs, users are entitled to rewards if they hold aTokens.

The `claimRewards` function is used to claim rewards from the rewards controller.

```
    function claimRewards(
        address[] calldata assets,
        uint256 amount,
        address to,
        address reward
    ) external override returns (uint256) {
        require(to != address(0), 'INVALID_TO_ADDRESS');
        return _claimRewards(assets, amount, msg.sender, msg.sender, to,
reward);
```

The AaveV3Adapter does not have a function to claim these rewards leaving them being stuck in the Aave.

Recommendations

Add a function to claim rewards from Aave `RewardsController`.

[M-06] Incorrect Increment in `_sendGainsToDepositor`

Severity

Impact:

Low

Likelihood:

High

Description

`_sendGainsToDepositor` uses the `continue` functionality to skip excess logic whenever `amount` to send is 0. But just before this, it increases `i` parameter in unchecked block. Under the hood, `continue` functionality also does this, as ideally it moves to the next `i`, so by previously increasing `i`, the next iteration is skipped.

For example, if we have an array of amounts [0,1,2,3]: In the first element, when `i` is 0, amount is 0; Since amount is 0, `i` is incremented via `i++` to 1; `continue` skips to the next element after `i`, which is 2. So the next iteration to be processed will be `i = 2` iteration.

This will lead to depositors missing out on gains from certain collaterals.

```
    function _sendGainsToDepositor(address _to, address[] memory assets,
uint256[] memory amounts) internal {
        uint256 assetsLen = assets.length;
        require(assetsLen == amounts.length, "StabilityPool: Length
mismatch");
        for (uint256 i = 0; i < assetsLen; ) {
            uint256 amount = amounts[i];
            if (amount == 0) {
                unchecked {
                    i++; //<@
                }
                continue;
            }
            address asset = assets[i];
            // Assumes we're internally working only with the wrapped
version of ERC20 tokens
            IERC20(asset).transfer(_to, amount);
            unchecked {
                i++;
            }
        }
    }
```

Recommendations

Remove the `unchecked` block and extraneous `i` increment.

[M-07] Single Adapter Failure Blocks All Yield Claims

Severity Medium

Impact:

Medium

Likelihood:

Medium

Description

The `YieldManager.sol` contract's `claimYield` function iterates through multiple yield adapters to collect rewards for a user. The current implementation lacks fault tolerance; if any single adapter's `claimYield` call reverts, the entire parent transaction fails. This creates a single point of failure where a bug, liquidity issue, or external dependency failure in one adapter can prevent a user from claiming their legitimate yield from all other functioning adapters.

The `claimYield` function is designed to aggregate yield from various external protocols by calling a corresponding adapter for each. The loop that executes these calls does not handle potential reverts from the external calls.

```
// Simplified example of the vulnerable loop
function claimYield(address _user) external {
    // ...
    for (uint i = 0; i < adapters.length; i++) {
        // This is a hard external call. If it reverts, the whole
transaction reverts.
        adapters[i].claimYield(_user);
    }
    // ...
}
```

If `adapters[i].claimYield(_user)` fails for any `i`—due to an adapter-specific bug, a problem in the underlying protocol (e.g., Aave), or insufficient liquidity for the claim—the entire `claimYield` transaction is reverted. The user is thus blocked from receiving yield from any of the other adapters, even those that are operating correctly.

Exploit Scenario

1. **User Deposits:** A user has supplied capital that is diversified across three different yield adapters managed by `YieldManager`.
2. **Adapter Failure:** One of the three external protocols (e.g., a specific lending pool) experiences a temporary issue. Its corresponding adapter's `claimYield` function now consistently reverts.
3. **DoS Condition:** The user calls `claimYield` on `YieldManager.sol`. The loop begins executing claims.
4. **Revert:** When the loop reaches the failing adapter, the external call reverts. This causes the entire `claimYield` transaction to revert.
5. **Impact:** The user cannot claim their accumulated yield from *any* of the three sources. The funds are not lost, but they are inaccessible until the single failing adapter is either fixed or removed from the manager, which may take an indefinite amount of time.

Recommendations

To build a more resilient system, wrap the external adapter calls within a `try/catch` block. This will isolate failures and allow the function to continue processing claims from the remaining healthy adapters. It is also recommended to emit an event when a failure is caught so that both the user and off-chain monitoring systems can be alerted to the specific adapter's issue.

[M-08] Snapshot Desynchronization in `withdraw()` Leads to Yield Theft

Severity Medium

Impact:

High

Likelihood:

Medium

Description

The `withdraw` function in `StabilityPool.sol` contains a flaw in its handling of depositor reward snapshots when called with `_claimYield` set to `false`. The function incorrectly updates parts of a user's snapshot to the current state while rolling back another part (`G`, the yield gain tracker) to a previous state. This creates a desynchronized, inconsistent snapshot. An attacker can leverage this inconsistent state to make a subsequent call to `claimYield()` that uses a corrupted formula, crediting them with an exponentially larger amount of yield than they are legitimately owed. This is a direct drain on the funds of other depositors and undermines the core economic safety of the stability pool.

Technical Description

The protocol uses a snapshot system to track each user's claim on rewards. The key values are `P` (a product accumulator that decreases with liquidations) and `G` (a gain accumulator that increases with yield). For the yield calculation to be correct, a user's `P` and `G` snapshots must always be recorded at the same point in time (i.e., same epoch and scale).

The `withdraw` function violates this invariant. When a user calls `withdraw` with `_claimYield = false`, the following sequence occurs:

1. The user's current `G` snapshot (`G0`) is stored in a local variable.
2. The function `_updateDepositAndSnapshots` is called, which correctly updates the user's *entire* snapshot to the current global values. The user's snapshot `P` is now `P1` (the current, lower value) and `G` is `G1` (the current, higher value).
3. The function then incorrectly overwrites just the `G` value of the user's snapshot with the stored `oldG` value: `depositorSnapshots[msg.sender].G = oldG;`.

This leaves the user with a corrupted snapshot: `(P=P1, G=G0)`. The snapshot pairs a new, low `P` value with an old, low `G` value, breaking the co-timestamping rule.

Exploit Scenario (Attack Path)

1. **Deposit:** An attacker deposits `D` tokens into the Stability Pool, receiving an initial, consistent snapshot (`P0`, `G0`).
2. **Wait:** The attacker waits for market activity. Liquidations cause the global `P` to decrease to `P1` (`P1 < P0`), and yield accrual causes the global `G` to increase to `G1` (`G1 > G0`).
3. **Manipulate Snapshot:** The attacker calls `withdraw(0, [], false)`. This transaction withdraws no funds but executes the flawed logic described above, setting their snapshot to the inconsistent state of (`P=P1`, `G=G0`).
4. **Claim Inflated Yield:** The attacker immediately calls `claimYield()`. The `getDepositorYieldGain` function calculates their reward using the corrupted snapshot. The formula, which is conceptually `yield = D * (G1 - G0) / P_snapshot`, uses `P1` in the denominator instead of the correct `P0`. Since `P1` is much smaller than `P0`, the resulting yield is artificially and massively inflated.
5. **Profit:** The inflated yield amount is transferred to the attacker from the pool's `yieldGainsOwed`, stealing funds that rightfully belong to other depositors.

Recommendations

The logic for `_claimYield = false` in the `withdraw` function must be fundamentally corrected to preserve the atomicity of snapshots.

[M-09] Linear approximation used for the fractional half-life period of the exponential decay calculation is inaccurate

Severity Medium

Impact: Medium

Likelihood: Medium

Description

This code calculates the remaining decayedRate using the formula: `decayedRate * (1 - remainingTime / (2 * halfLifeSeconds))`

```
// For partial half-life periods, use linear approximation
if (remainingTime > 0 && decayedRate > 0) {
    uint256 partialDecay = (decayedRate * remainingTime) / (2 *
halfLifeSeconds);
    decayedRate = decayedRate > partialDecay ? decayedRate - partialDecay
: 0;
}
```

Let's define `x = remainingTime / halfLifeSeconds` , where 0 < x < 1 . The formula becomes: `decayedRate * (1 - 0.5x)`

This is a linear approximation of the true exponential decay formula, which is: `decayedRate * (0.5^x)` or `decayedRate * 2^(-x)`

The linear approximation 1 - 0.5x is not a precise representation of the exponential `curve 2^(-x)` . Let's compare them:

- At x = 0 (0 seconds elapsed), both formulas correctly yield 1 .
- At x = 1 (a full half-life has passed), both formulas correctly yield 0.5 .
- However, for any point between 0 and 1, the linear approximation overestimates the true value. For example, at `x = 0.5` (12 hours):
    - Linear Approximation: `1 - 0.5 * 0.5 = 0.75`
    - Actual Value: `2^(-0.5) ≈ 0.707` The approximation consistently calculates a higher decayedRate than the actual value.

PoC Sketch

- Half-life=86400s; time=129600s (1.5 days); base=1e18.
- Exact: decayed = 1e18 * (0.5)^1.5 ≈ 0.3535e18.
- Code: Halves to 0.5e18, linear partial (0.5e18 - (0.5e18 * 43200 / (286400))) = 0.5e18 - 0.125e18 = 0.375e18 (5.9% overestimate).
- Scaled: For base=100e18, error=2.2e18, affecting fees by millions in value.

Recommendations

Instead of Linear Approximation use Quadratic Approximation

[M-10] Violation of Economic Invariant

Severity Medium

Impact:

High

Likelihood:

Low

Description

The core promise violated here is vault solvency post-creation: A leveraged vault should open with a `collateralization ratio (CR) ≥ MCR + buffer`, ensuring protocol-wide over-collateralization. If a swap yields less collateral than expected (due to manipulated liquidity), the vault starts under-collateralized, allowing immediate liquidation for attacker profit. This creates a profitable extraction vector, desynchronizing internal state (vault CR) from economic reality (true collateral value).

Root Cause:

1. **Pre-Swap Debt Minting Without Liquidity Lock** : Debt (KEI) is minted and transferred to `SwapRouter` before the swap, based on the leverage formula. This creates a fixed debt obligation that doesn't adjust if the swap underperforms due to external factors.

2. **Vulnerability to Concurrent State Changes** : Liquidations trigger `StabilityPool.offsetDebt` (`offsetDebt`), which burns KEI and distributes collateral. In a KEI-collateral DEX pair, this burns circulating KEI, potentially spiking its price and reducing effective liquidity for the victim's swap. An attacker can front-run the `createLeveragedVault` transaction by triggering a liquidation, forcing the swap to execute at a worse rate (more KEI per unit of collateral).

3. **Inadequate Post-Swap Validation** : SwapRouter 's `_validateSwapPrice` checks deviation against a pre-swap oracle price with `MAX_PRICE_DEVIATION` . However, if the manipulated slippage falls within this threshold (e.g., 1-2%), the swap proceeds, but the `finalCollateralAmount` is insufficient for the fixed debtAmount, resulting in `CR < MCR`. The vault is created anyway, as the check in the swap Mechanism only verifies against the post-swap amount without re-evaluating the leverage assumption.

4. HyperEVM-Specific Amplification : With sub-second block times (~0.98s) , the mempool is highly manipulable, making sandwich attacks economically viable. Attackers profit by liquidating the under-collateralized vault immediately, extracting the liquidation penalty while the protocol absorbs insolvency risk.

Impact

- Attacker Profit : For a $100K leveraged position at 2x, a 1% slippage manipulation could create ~$1K under-collateralization, yielding the attacker a 10% liquidation bonus ($100) minus gas fees.
- Protocol Loss : Cumulative under-collateralized vaults erode reserves, potentially leading to systemic insolvency if bad debt accumulates.

Profitability

1. User submits openVault with leverage, routing KEI to `SwapRouter` for collateral swap.
2. Attacker (or concurrent liquidation) calls `liquidateVaults`, triggering `offsetDebt` in `StabilityPool`, which burns KEI and adds collateral to the pool, draining DEX liquidity.
3. Swap executes at worse rate (e.g., user gets 5% less collateral due to sudden scarcity).
4. Post-validation passes if within `MAX_PRICE_DEVIATION`, but vault opens with `CR = (initial + received) / debt` below safe threshold (e.g., 105% vs. required 110% + buffer), making it instantly liquidatable for profit.

Quantifiable Profit: For $1M, 2% slippage yields $20K under-collateralization. Attacker profits from liquidation penalty (~$10K if 10% on $100K debt portion) minus gas (negligible on HyperEVM's low fees). Scales with TVL; in high-volatility scenarios, multiple sandwiches could drain very high amount

Recommendation

- **Pre-Swap Liquidity Check**: Snapshot Stability Pool's `totalDebtTokenDeposits` and DEX reserves before swap; abort if `deviation > threshold`.

- **Atomic Bundling**: Use flash loans or bundle liquidation checks into vault creation to prevent interleaving.

[L-01] Yield Distribution Percentage Sum Not Checked

Contract: `VaultOperations.sol`:`setStakingModuleYieldPercentage(uint256)`, `setStabilityPoolYieldPercentage(uint256)`

Description

The `setStakingModuleYieldPercentage` and `setStabilityPoolYieldPercentage` functions check that percentages cannot be set more than 100% individually but do not check if the sum of `stakingModuleYieldPercentage` and `stabilityPoolYieldPercentage` exceeds 100%.

```
  function setStakingModuleYieldPercentage(uint256 _percentage) external
onlyOwner {
        if (_percentage > 10000) revert PercentageCannotExceed100();
        stakingModuleYieldPercentage = _percentage;
        emit StakingModuleYieldPercentageUpdated(_percentage);
    }

    /**
     * @notice Sets the percentage of interest that goes to the stability
pool (in basis points)
     * @param _percentage Percentage in basis points (e.g., 3000 for 30%)
     */
    function setStabilityPoolYieldPercentage(uint256 _percentage) external
onlyOwner {
        if (_percentage > 10000) revert PercentageCannotExceed100();
        stabilityPoolYieldPercentage = _percentage;
        emit StabilityPoolYieldPercentageUpdated(_percentage);
    }
```

Impact

If the owner sets the combined percentages to more than 100%, calls to `distributeYield` will revert due to an arithmetic underflow. This creates a temporary Denial of Service (DoS) for protocol revenue distribution, which can be fixed by the owner.

Recommendation

In both setter functions, add a check to ensure the sum of the two percentages does not exceed 10,000 (100%).

Example:

```
// In setStakingModuleYieldPercentage
if (_percentage + stabilityPoolYieldPercentage > 10000) revert
```

```
TotalPercentageExceeds100();

// In setStabilityPoolYieldPercentage
if (stakingModuleYieldPercentage + _percentage > 10000) revert
TotalPercentageExceeds100();
```

[L-02] Unbounded Withdrawal Requests in `StakingModule`

Description

Withdrawal requests are never deleted, only marked as claimed. This leads to unbounded array growth, which can cause out-of-gas errors for users with many requests.

Impact: Potential DoS or out of gas errors for users with many withdrawal requests.

Recommendations

Either Remove the Requests once processed or Put an upper limit on the withdrawal request limit per user

[L-03] Incorrect Negative Collateral Check

Description

`forceRetrieveCollateral` Checks if (collateralAmount < 0) on a uint256, which is always false as an unsigned integer value can never be less than zero

```
function forceRetrieveCollateral(address vaultCollateral, address
vaultOwner) external returns (uint256) {

// ...existing Code


@>  if (collateralAmount < 0) revert VaultDoesntExist();

// .. remaining code

## Recommendations
Change the check to either equivalent to zero or any other value that
corresponds with the business Logic
```

[L-04] Missing Min Debt Amount Check in Various Cases

Description

Create vault ensures that the debt amount is greater than the minimum debt amount, but this is not enforced in other functions like `createLeveragedVault` and `adjustVault`.

```
    function createLeveragedVault(
        address vaultCollateral,
        uint256 initialCollateralAmount,
        uint256 targetLeverageRatio,
        uint256 interestRate,
        uint256 maxSlippageBps,
        address[] calldata tokens,
        ISwapRouter.Swap[][] calldata hopSwaps,
        address prevId,
        address nextId
    ) external {
//...

        // Check mint cap before proceeding
        if (totalDebt[vaultCollateral] + debtAmount >
IVaultManager(vaultManager).getMintCap(vaultCollateral)) revert
MaxDebtExceeded();

        // Update protocol accounting
        totalDebt[vaultCollateral] += debtAmount;
        totalProtocolDebt += debtAmount;
        activeVaults += 1;

        // Mint KEI to SwapRouter so it can execute the swap
        IERC20(debtToken).mint(swapRouter, debtAmount);//<@

//...
```

```
    function adjustVault(
        address vaultCollateral,
        uint256 addCollateral,
        uint256 withdrawCollateral,
        uint256 addDebt,
        uint256 repaidDebt,
        address prevId,
        address nextId
    ) external {
//...

        uint256 newDebt = debtAmount + addDebt - repaidDebt;//<@

        // BORROWING THRESHOLD CHECK - Only when increasing debt or
withdrawing collateral
        if (addDebt > 0 || withdrawCollateral > 0) {
            uint256 newCollateral = collateralAmount + addCollateral -
```

```
withdrawCollateral;

            // Only check if there will be debt after the operation
            if (newDebt > 0) {
                require(

IVaultManager(vaultManager).checkBorrowingAllowed(vaultCollateral,
msg.sender, newCollateral, newDebt),
                    "Operation would result in CR below borrowing
threshold"
                );
            }
        }

        IVaultManager(vaultManager).adjustVaultData(vaultCollateral,
msg.sender,
            collateralAmount + addCollateral - withdrawCollateral,
            newDebt,
            interestRate);//<@

//...
    }
```

The downsides of this is that vaults can exist with debt amount less than min debt amount, sometimes as low as a few token wei. These vault types are not really useful as there's no incentive to liquidate them if they ever go bad. Another potential downside is that these vault take up space in the set of available vaults, which can be a problem in a function like `_executeCollateralRedemption` in `redeemVault` which iterates over available vaults to redeem from. If there's enough of these vaults, an out of gas error can occur.

```
    function _executeCollateralRedemption(
        address collateral,
        uint256 keiAmount,
        uint256 feeRate,
        address prevId,
        address nextId
    ) internal returns (CollateralBundle memory bundle) {
        bundle.collateralToken = collateral;
        uint256 remainingKei = keiAmount;

        // Start from lowest ARS (riskiest) vaults in this collateral
        address currentVault =
IVaultSorter(vaultSorter).getLast(collateral);

        while (remainingKei > 0 && currentVault != address(0)) {//<@
            // Update vault interest and get data
            (uint256 collateralAmount, uint256 debtAmount,) =

IVaultOperations(vaultOperations).updateVaultInterest(collateral,
currentVault);
```

```
            if (debtAmount == 0) {
                currentVault =
IVaultSorter(vaultSorter).getPrev(collateral, currentVault);
                continue;
            }

            // Calculate redemption amounts - consolidated to reduce stack
depth
            uint256 redeemableKei = debtAmount < remainingKei ? debtAmount
: remainingKei;
            uint256 netRedeemableKei = redeemableKei - (redeemableKei *
feeRate) / DECIMAL_PRECISION;
            uint256 collateralToRedeem = (netRedeemableKei *
DECIMAL_PRECISION) / 6e18; // Fixed price for now

            // Adjust if insufficient collateral
            if (collateralAmount < collateralToRedeem) {
                collateralToRedeem = collateralAmount;
                netRedeemableKei = (collateralToRedeem * 6e18) /
DECIMAL_PRECISION;
                redeemableKei = (netRedeemableKei * DECIMAL_PRECISION) /
(DECIMAL_PRECISION - feeRate);
            }

            // Execute redemption and update totals
            _redeemFromSingleVault(collateral, currentVault,
redeemableKei, collateralToRedeem, prevId, nextId);
            bundle.collateralAmount += collateralToRedeem;
            bundle.keiAmount += netRedeemableKei;
            remainingKei -= redeemableKei;

            currentVault = IVaultSorter(vaultSorter).getPrev(collateral,
currentVault);
        }

        return bundle;
    }
```

Recommendations

Ensure that vault debt is never less than min debt amount across protocol.

[L-05] Inconsistent Total Protocol Debt Tracking

Description

The protocol uses two different methods to calculate the "total protocol debt," leading to inconsistencies that can affect critical system operations like redemptions and fee calculations.

1. **In `redeemVault`:** The total protocol debt is fetched as the total supply of the debt token (`IERC20(debtToken).totalSupply()`).
2. **In `calculateRedemptionFee`:** The total protocol debt is calculated as the sum of the debt across all vaults for all valid collateral types (`IVaultOperations(vaultOperations).totalDebt(collaterals[i])`).
3. **In VaultOperations.sol:** The total protocol debt is also declared as a parameter which is constantly updated during protocol operations.

These values should always be equal, as minting and burning debt tokens is directly tied to creating and repaying vault debt. However, if there is ever a discrepancy (e.g., due to a bug in a mint/burn operation, a special privileged function, or a precision error), it will cause incorrect calculations.

- **In `redeemVault`:** An inflated `totalSupply` would allow a redemption to burn more KEI than the actual debt in the system.
- **In `calculateRedemptionFee`:** An inaccurate sum would lead to incorrect fee and base rate calculations.

This inconsistency introduces unnecessary risk into core protocol mechanics.

Recommendations

Use a single source of truth for the total protocol debt throughout the system to ensure consistency.

[L-06] Changing `withdrawCooldown` impact pending withdrawals

Description

`claimWithdrawnSKEIKO` function uses current `withdrawCooldown` to ensure that the withdrawal cooldown period is met before a user can claim their withdrawal. This is compared based on the `requestTime` of the withdrawal request. If cooldown is changed between the time of request and the time of claim, the user's claim period might be extended or shortened depending on the cooldown value set.

```
    function setWithdrawCooldown(uint256 _cooldown) external onlyOwner {
        withdrawCooldown = _cooldown;//<@
        emit UnstakeCooldownUpdated(_cooldown);
    }
```

Recommendations

A potential fix is to use a parameter to track expected endtime as at the time of creating the request. It can be added to the `WithdrawRequest` struct. The during request claims, current time will be compared with the expected endtime instead.

[L-07] Immutable Aave Pool Address

Description

Aave pool is set as immutable in AaveV3Adapter's constructor which goes against Aave's recommendation to always query the pool address dynamically whenever it is needed. This is because potential pool upgrades/migration can occur, which will break any immutable implementation.

```
    IPool public immutable aavePool;//<@
    IPoolAddressesProvider public immutable addressesProvider;
    address public immutable yieldManager;
```

AAVE has updated pool addresses in the past (e.g., during major version upgrades like V2 to V3 or pool migrations) so pool migrations is a realistic scenario, especially over the long term as AAVE evolves.

Recommendations

Query the pool address from the `addressesProvider` dynamically whenever it is needed.

[L-08] `emergencyWithdraw`'s Catch Logic Could be Dangerous

Description

`emergencyWithdraw` passes in `type(uint256).max` as the amount to withdraw from aave. This means all of adapter's atoken balance which includes atokens from multiple different users. The entire withdrawn amount is then sent to the recipient. If for any reason the `try` block fails, the contract's entire atoken balance is withdrawn, sending all deployed amount from all users to the recipient. Odds of this happening are very low, but as a precaution.

```
    function emergencyWithdraw(address asset, uint256 amount, address
recipient) external override onlyYieldManager returns (uint256) {
        address aToken = assetToAToken[asset];

        if (aToken == address(0)) {
            IPool.ReserveData memory reserveData =
aavePool.getReserveData(asset);
            aToken = reserveData.aTokenAddress;
        }

        uint256 aBalance = IERC20(aToken).balanceOf(address(this));
        if (aBalance == 0) return 0;

        uint256 toWithdraw = amount > aBalance ? aBalance : amount;

        try aavePool.withdraw(asset, toWithdraw, address(this)) returns
(uint256 withdrawn) {
            IERC20(asset).transfer(recipient, withdrawn);
            emit EmergencyWithdrawal(asset, withdrawn, recipient);
```

```
            return withdrawn;
        } catch {
            // Try max withdrawal
            try aavePool.withdraw(asset, type(uint256).max, address(this))
returns (uint256 withdrawn) {//<@
                IERC20(asset).transfer(recipient, withdrawn);//<@
                emit EmergencyWithdrawal(asset, withdrawn, recipient);
                return withdrawn;
            } catch {
                return 0;
            }
        }
    }
```

Recommendations

In the catch block, instead of sending the withdrawn amount, send how much of it the recipient is supposed to receive, otherwise, just return 0 instead of max withdrawal.

[L-09] Rate Adjustments with 0 `targetDebtAhead` can be created

Description

In `toggleRateAdjustment`, if the `enabled` flag is set to `true`, it ensures that the `targetDebtAhead` is greater than 0. The function also ensures that `targetDebtAhead` to set is greater than 0. But in `configureAndEnableRateAdjustment`, the finally calculated `effectiveTargetDebtAhead` can be 0 if the `totalDebtForCollateral` == `userDebt`. Whatever `targetDebtAhead` will be greater than the `maxPossibleDebtAhead`. As a result, even when the `enabled` flag can be set to `true` though `targetDebtAhead` is 0.

```
    function configureAndEnableRateAdjustment(
        address collateral,
        uint256 targetDebtAhead,
        uint256 bufferPercentage,
        uint256 feePercentage
    ) external {
        (uint256 collateralAmount, , ) =
IVaultManager(vaultManager).getVaultData(collateral, msg.sender);
        require(collateralAmount > 0, "No vault exists");

        require(targetDebtAhead > 0, "Target debt ahead must be greater
than 0");//<@
        require(bufferPercentage <= 10000, "Buffer percentage too high");
// Max 100%
        require(feePercentage <= 10000, "Fee percentage too high"); // Max
100%
```

```
        // Get total debt for collateral and user's debt
        uint256 totalDebtForCollateral =
IVaultOperations(vaultOperations).totalDebt(collateral);
        (, uint256 userDebt, ) =
IVaultManager(vaultManager).getVaultData(collateral, msg.sender);

        // Maximum possible debt ahead is total debt minus user's own debt
        uint256 maxPossibleDebtAhead = totalDebtForCollateral -
userDebt;//<@

        // Cap target debt ahead at maximum possible (making vault the
safest if target exceeds max)
        uint256 effectiveTargetDebtAhead = targetDebtAhead;
        if (targetDebtAhead > maxPossibleDebtAhead) {
            effectiveTargetDebtAhead = maxPossibleDebtAhead;//<@
        }

        RateAdjustmentConfig storage config =
adjustmentConfigs[msg.sender][collateral];

        config.targetDebtAhead = effectiveTargetDebtAhead;
        config.bufferPercentage = bufferPercentage;
        config.feePercentage = feePercentage;
        config.enabled = true;

        emit RateAdjustmentConfigured(msg.sender, collateral,
effectiveTargetDebtAhead, bufferPercentage, feePercentage);
        emit RateAdjustmentEnabled(msg.sender, collateral);
    }
```

Recommendations

Add a check that `effectiveTargetDebtAhead` is not 0 before setting the `enabled` flag to `true`.

[I-01] Missing `mint` and `burn` Functionalities

Description

KEIKO.sol is missing `mint` and `burn` functionalities, and it is not possible to mint or burn tokens from the contract. The constructor also hasn't minted tokens to needed addresses.

```
    constructor(address stakingModule) ERC20(NAME, SYMBOL, DECIMALS) {
        require(stakingModule != address(0), "Invalid staking module");

        transferFromWhitelisted[stakingModule] = true;

        /* TODO: mint tokens to specific addresses following TGE
tokenomics
            _mint(address(0), 0 * 1e18);
```

```
        */
    }
```

Recommendations

Add `mint` and `burn` functionalities to KEIKO.sol or mint tokens to needed addresses in constructor.

[I-02] Redundant Code

Description

1. `forceRetrieveCollateral` for no reason approves the yield manager to spend all vault collateral tokens. But the approval is not used in subsequent operations, or in other parts of YieldManager at all.

```
    function forceRetrieveCollateral(address vaultCollateral, address
vaultOwner) external returns (uint256) {
//...
        IERC20(vaultCollateral).approve(yieldManager, type(uint256).max);
//<@
        uint256 retrievedAmount =
IYieldManager(yieldManager).forceRetrieveCollateral(vaultCollateral,
vaultOwner, msg.sender);

        emit ForceRetrievalExecuted(vaultOwner, vaultCollateral,
msg.sender, retrievedAmount);

        return retrievedAmount;
    }
```

2. `executeSwapAndReturnCollateral` can only be called by `VaultOperations` via the `createLeveragedVault` function which sends `initialCollateralAmount` of collateral to swap to the router. So the check that router balance is greater than or equal to `initialCollateralAmount` is redundant as it will always pass.

```
    function executeSwapAndReturnCollateral(
    function executeSwapAndReturnCollateral(
        address collateral,
        uint256 initialCollateralAmount,
        uint256 debtAmount,
        uint256 maxSlippageBps,
        address[] calldata tokens,
        ISwapRouter.Swap[][] calldata hopSwaps
    ) external onlyVaultOperations returns (uint256 totalCollateral) {
        if (tokens.length < 2 || tokens[0] != debtToken ||
tokens[tokens.length - 1] != collateral) {
```

```
            revert InvalidSwapPath();
        }

        uint256 collateralBalance =
IERC20(collateral).balanceOf(address(this));
        if (collateralBalance < initialCollateralAmount) {//<@
            revert InsufficientCollateral();
        }
//...
        return totalCollateral;
    }
```

3. `configureAndEnableRateAdjustment` calls `IVaultManager(vaultManager).getVaultData` twice, once for collateral amount and once for user debt without making any state changes between these calls. Calling the second time is not needed.

```
    function configureAndEnableRateAdjustment(
        address collateral,
        uint256 targetDebtAhead,
        uint256 bufferPercentage,
        uint256 feePercentage
    ) external {
        (uint256 collateralAmount, , ) =
IVaultManager(vaultManager).getVaultData(collateral, msg.sender);//<@
        require(collateralAmount > 0, "No vault exists");

        require(targetDebtAhead > 0, "Target debt ahead must be greater
than 0");
        require(bufferPercentage <= 10000, "Buffer percentage too high");
// Max 100%
        require(feePercentage <= 10000, "Fee percentage too high"); // Max
100%

        // Get total debt for collateral and user's debt
        uint256 totalDebtForCollateral =
IVaultOperations(vaultOperations).totalDebt(collateral);
        (, uint256 userDebt, ) =
IVaultManager(vaultManager).getVaultData(collateral, msg.sender);//<@

//...
    }
```

4. `createVault` calls `checkVaultState` and reverts if the function doesn't return true. However, the function only returns true if all of the internal checks pass. It never returns false, so this check is redundant. Just call `checkVaultState` without checking for the return value.

```
    function createVault(address vaultCollateral, uint256 collatAmount,
uint256 debtAmount, uint256 interestRate, address prevId, address nextId)
```

```
external {
//...
        IVaultManager(vaultManager).adjustVaultData(vaultCollateral,
msg.sender, collatAmount, debtAmount, interestRate);
        if (!IVaultManager(vaultManager).checkVaultState(vaultCollateral,
msg.sender)) revert InvariantCheckFailed();//<@
        if (totalDebt[vaultCollateral] >
IVaultManager(vaultManager).getMintCap(vaultCollateral)) revert
MaxDebtExceeded();

        uint256 vaultARS =
IVaultManager(vaultManager).calculateARS(vaultCollateral, msg.sender);
        IVaultSorter(vaultSorter).insertVault(vaultCollateral, msg.sender,
vaultARS, prevId, nextId);

        IERC20(vaultCollateral).transferFrom(msg.sender, address(this),
collatAmount);
        IERC20(debtToken).mint(msg.sender, debtAmount);

        emit VaultCreated(msg.sender, vaultCollateral, collatAmount,
debtAmount);
    }
```

The same can be seen in `deployCollateralToYield` which calls `deployCollateral` and checks that the return value is not false even though the function never returns false.

```
    function deployCollateralToYield(address vaultCollateral, uint256
amount, address adapter) external {
        (uint256 collateralAmount, , ) =
manageDebtInterest(vaultCollateral, msg.sender);

        if (collateralAmount == 0) revert VaultDoesntExist();
        if (amount == 0) revert AmountMustBeGreaterThanZero();
        if (amount > collateralAmount) revert
AmountExceedsAvailableCollateral();

        IERC20(vaultCollateral).approve(yieldManager, amount);

        // Deploy collateral to yield strategy
        bool success = IYieldManager(yieldManager).deployCollateral(//<@
            vaultCollateral,
            msg.sender,
            amount,
            adapter
        );

        if (!success) revert DeploymentFailedBufferRequirements();

        emit CollateralDeployedToYield(msg.sender, vaultCollateral,
amount);
    }
```

Recommendations

Remove or refactor the code as needed.